LAPORAN TUGAS AKHIR

MODUL 4 - Inheritance, Polymorphism, dan Enkapsulasi



Disusun Oleh:

Akmallutfi Afza Gusty

707012500064

SIKC 49 - 04

Mata Kuliah:

Dasar Pemrograman Perangkat Bergerak

Dosen Pengampu:

Asep Hadayana

1. Tujuan Praktikum

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep inheritance (pewarisan) dan penerapannya pada Dart.

2. Membuat class anak dengan kata kunci extends.

3. Menerapkan method overriding menggunakan annotation @override.

4. Menerapkan enkapsulasi melalui field privat dan getter.

5. Menggunakan constructor class induk pada class anak dengan kata kunci super.

2. Alat dan Bahan

Untuk melaksanakan praktikum pemrograman menggunakan bahasa Dart, diperlukan perangkat dan perangkat lunak pendukung sebagai berikut:

1. Perangkat Keras (Hardware)

a. Laptop

b. Koneksi internet untuk mengunduh Dart SDK dan mengakses dokumentasi.

2. Perangkat Lunak (Software)

a. Sistem operasi Windows 11 Home Single Language (64-bit).

b. Dart SDK versi 3.13.4 (stable) yang mencakup kompiler Dart dan Dart VM.

c. Editor kode Visual Studio Code dengan ekstensi Dart, atau DartPad untuk percobaan ringan.

3. Langkah-Langkah Praktikum

Pada praktikum ini, program dibuat menggunakan bahasa Dart untuk mengimplementasikan konsep OOP berupa inheritance, method overriding, dan enkapsulasi. Studi kasusnya adalah agen ekspedisi CepatKirim yang menerima paket, menghitung ongkos kirir berdasarkan jenis layanan, lalu mencatat total pendapatan. Seluruh kode program ditulis dalam satu berkas bernama ta.dart. Berikut langkah-langkah pembuatannya.

3.1. Membuat Class Paket sebagai Class Induk

Langkah pertama adalah mendeklarasikan class Paket sebagai class induk. Class ini memiliki tiga atribut, yaitu namaPengirim (String), tujuan (String), dan beratKg (double). Constructor-nya memakai shorthand this.namaPengirim sehingga nilai parameter langsung diberikan ke atribut yang bersangkutan. Class Paket juga memiliki method hitungOngkir()

yang mengembalikan nilai 0. Method ini menjadi perhitungan awal yang nantinya diganti oleh class anak.

```
class Paket {
  String namaPengirim;
  String tujuan;
  double beratKg;

  Paket(this.namaPengirim, this.tujuan, this.beratKg);

  hitungOngkir() => 0;
}
```

3.2. Membuat Class Anak PaketReguler dan PaketKilat

Pewarisan diterapkan dengan kata kunci extends. Class PaketReguler dan PaketKilat sama-sama mewarisi atribut dan constructor dari class Paket. Kedua class anak menimpa method hitungOngkir() dengan annotation @override. Tarif paket reguler dihitung dari berat dikali 10.000, sedangkan paket kilat dihitung dari berat dikali 15.000 lalu ditambah biaya layanan 5.000. Constructor masing-masing class anak meneruskan argumen ke class induk menggunakan super.namaPengirim, super.tujuan, dan super.beratKg.

```
class PaketReguler extends Paket {
  PaketReguler(super.namaPengirim, super.tujuan, super.beratKg);
  @override
  hitungOngkir() {
    return beratKg * 10000;
  }
}

class PaketKilat extends Paket {
  PaketKilat(super.namaPengirim, super.tujuan, super.beratKg);
  @override
  hitungOngkir() {
    return (beratKg * 15000) + 5000;
  }
}
```

3.3. Membuat Class AgenEkspedisi

Class AgenEkspedisi berperan sebagai pengelola paket. Atribut daftarPaket bertipe List<Paket> menyimpan seluruh paket yang diterima. Total pendapatan disimpan dalam

field privat _totalPendapatan sehingga tidak dapat diubah langsung dari luar class. Nilainya hanya bisa dibaca melalui getter cekPendapatan. Method terimaPaket() menambahkan paket ke daftarPaket, menambahkan hasil hitungOngkir() ke _totalPendapatan, lalu mencetak konfirmasi bahwa paket telah diterima.

```
class AgenEkspedisi {
  List<Paket> daftarPaket = [];

  double _totalPendapatan = 0;

  double get cekPendapatan => _totalPendapatan;

  void terimaPaket(Paket p) {
    daftarPaket.add(p);
    _totalPendapatan += p.hitungOngkir();

    print("Paket tujuan ${p.tujuan} berhasil diterima agen");
  }
}
```

3.4. Membuat Menu Utama di fungsi main()

Fungsi main() membuat objek agenPusat bertipe AgenEkspedisi, kemudian menampilkan menu di dalam perulangan while (true) agar pengguna dapat bertransaksi berulang kali. Input dibaca dengan stdin.readLineSync(). Pada menu 1, program meminta nama pengirim, tujuan, berat, dan jenis layanan, lalu membuat objek PaketReguler atau PaketKilat sesuai pilihan. Menu 2 menampilkan semua paket dengan perulangan for-in. Menu 3 menampilkan total pendapatan melalui getter cekPendapatan. Menu 4 menghentikan perulangan dengan break. Karena fungsi main() cukup panjang, kodenya ditampilkan dalam dua bagian gambar berikut.

```
void main() {
  var agenPusat = AgenEkspedisi();

  while (true) {
    print("\n=== AGEN EKSPEDISI CEPATKIRIM ===");
    print("1. Terima Paket Baru");
    print("2. Lihat Semua Paket");
    print("3. Cek Total Pendapatan Agen");
    print("4. Keluar");
    stdout.write("Pilih menu (1-4): ");

    String pilihan = stdin.readLineSync() ?? "";

    if (pilihan == "1") {
      stdout.write("Nama Pengirim: ");
      String nama = stdin.readLineSync() ?? "";
      stdout.write("Tujuan: ");
      String tujuan = stdin.readLineSync() ?? "";
      stdout.write("Berat (Kg): ");
      double berat = double.parse(stdin.readLineSync() ?? "0");
      stdout.write("Jenis Layanan (1: Reguler, 2: Kilat): ");
      String jenis = stdin.readLineSync() ?? "";

      if (jenis == "1") {
        var p = PaketReguler(nama, tujuan, berat);
        agenPusat.terimaPaket(p);
      } else if (jenis == "2") {
        var p = PaketKilat(nama, tujuan, berat);
        agenPusat.terimaPaket(p);
      } else {
        print("Pilihan layanan tidak valid!");
      }
    }
```

```
    else if (pilihan == "2") {
      print("\n--- DAFTAR PAKET MASUK ---");
      for (var p in agenPusat.daftarPaket) {
        print("Pengirim: ${p.namaPengirim}, Tujuan: ${p.tujuan}");
      }
    } else if (pilihan == "3") {
      print("\nTotal Pendapatan: Rp ${agenPusat.cekPendapatan}");
    } else if (pilihan == "4") {
      print("Menutup sistem... Terima kasih!");
      break;
    } else {
      print("Menu tidak tersedia!");
    }
  }
}
```

4. Hasil dan Dokumentasi Program

Program dijalankan melalui terminal dengan perintah dart ta.dart. Contoh sesi berikut menggunakan dua paket, yaitu paket reguler 5 kg tujuan Jakarta dan paket kilat 2 kg tujuan Bandung.

Gambar 4.1. Input Paket Reguler

```
=== AGEN EKSPEDISI CEPATKIRIM ===
1. Terima Paket Baru
2. Lihat Semua Paket
3. Cek Total Pendapatan Agen
4. Keluar
Pilih menu (1-4): 1
Nama Pengirim: Budi Santoso
Tujuan: Jakarta
Berat (Kg): 5
Jenis Layanan (1: Reguler, 2: Kilat): 1
Paket tujuan Jakarta berhasil diterima agen
```

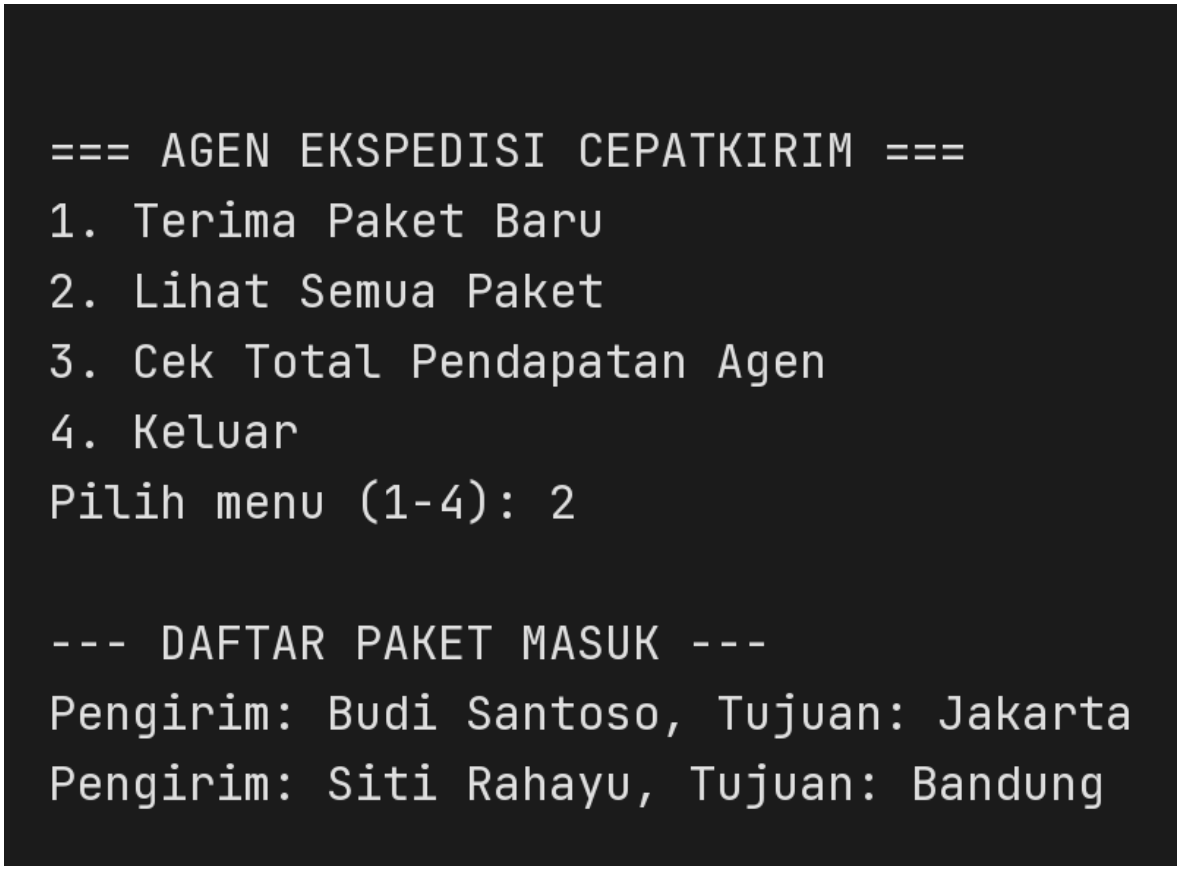
Keterangan Gambar 4.1: Pengguna memilih menu 1 lalu mengisi data pengirim Budi Santoso dengan tujuan Jakarta dan berat 5 kg. Layanan reguler dipilih dengan mengetik 1 pada pilihan jenis layanan. Program mencetak konfirmasi bahwa paket tujuan Jakarta berhasil diterima agen.

Gambar 4.2. Input Paket Kilat

```
=== AGEN EKSPEDISI CEPATKIRIM ===
1. Terima Paket Baru
2. Lihat Semua Paket
3. Cek Total Pendapatan Agen
4. Keluar
Pilih menu (1-4): 1
Nama Pengirim: Siti Rahayu
Tujuan: Bandung
Berat (Kg): 2
Jenis Layanan (1: Reguler, 2: Kilat): 2
Paket tujuan Bandung berhasil diterima agen
```

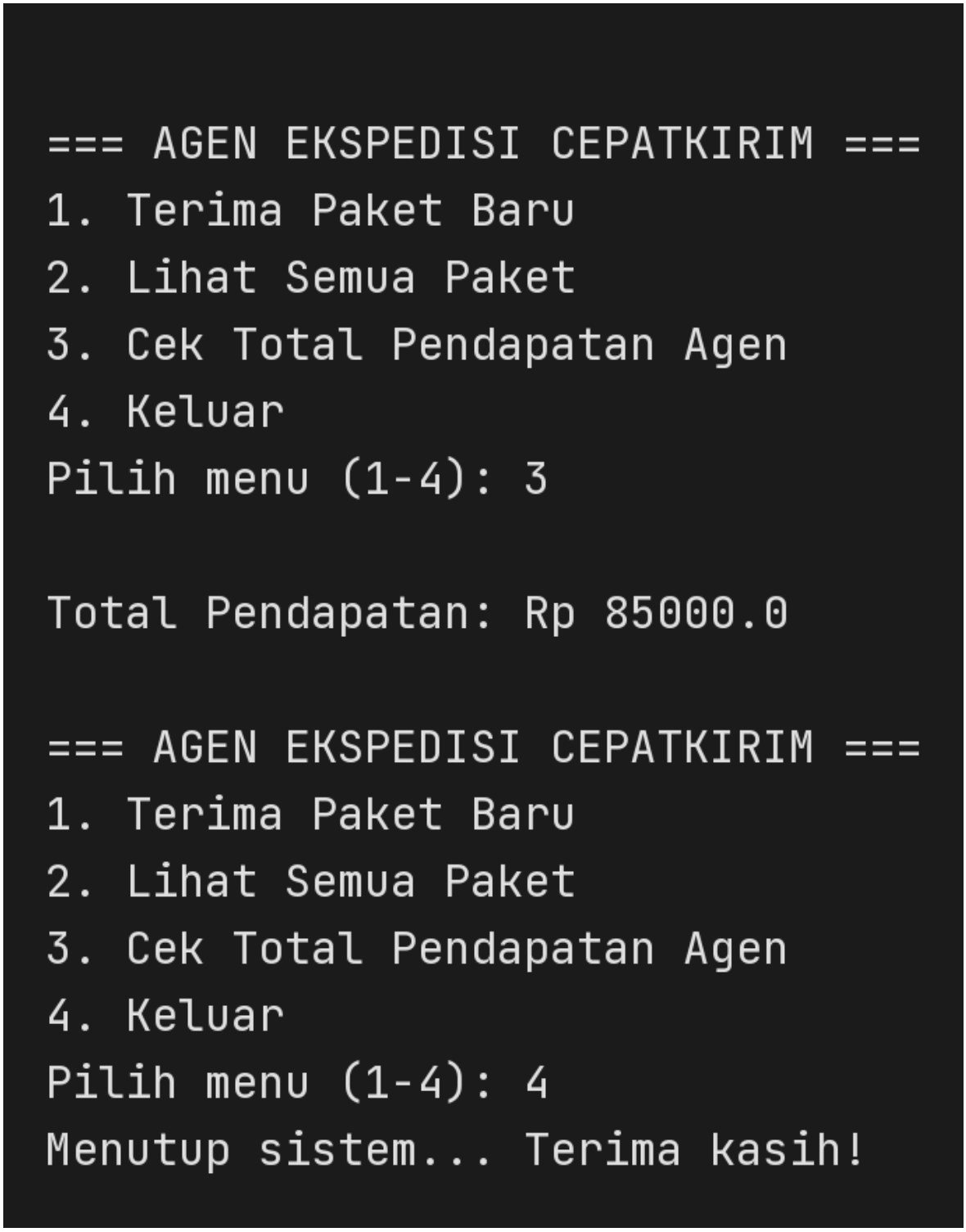
Keterangan Gambar 4.2: Pengguna mengisi data pengirim Siti Rahayu dengan tujuan Bandung dan berat 2 kg, kemudian memilih layanan kilat dengan mengetik 2. Program kembali mencetak konfirmasi penerimaan paket tujuan Bandung.

Gambar 4.3. Menampilkan Semua Paket



Keterangan Gambar 4.3: Menu 2 menampilkan daftar paket yang telah diterima. Kedua paket tersimpan di dalam daftarPaket lengkap dengan nama pengirim dan tujuannya masing-masing.

Gambar 4.4. Cek Total Pendapatan dan Keluar



Keterangan Gambar 4.4: Menu 3 menampilkan total pendapatan agen sebesar Rp 85000.0. Angka tersebut berasal dari paket reguler (5 kg x Rp 10.000 = Rp 50.000) ditambah paket kilat (2 kg x Rp 15.000 + Rp 5.000 = Rp 35.000). Menu 4 menutup program dengan pesan terima kasih.

5. Analisis dan Pembahasan

- Penerapan inheritance: class PaketReguler dan PaketKilat mewarisi atribut namaPengirim, tujuan, dan beratKg dari class Paket, sehingga ketiga atribut tidak perlu dideklarasikan ulang di class anak.
- Method overriding: kedua class anak menimpa method hitungOngkir() dengan @override. Pemanggilan p.hitungOngkir() di dalam terimaPaket() tetap ditulis berdasarkan tipe data Paket, tetapi method yang dieksekusi disesuaikan dengan objek aslinya. Inilah wujud polimorfisme pada program ini.
- Perhitungan ongkos kirir sesuai tarif masing-masing layanan. Paket reguler 5 kg menghasilkan 5 x 10.000 = Rp 50.000, sedangkan paket kilat 2 kg menghasilkan (2 x 15.000) + 5.000 = Rp 35.000. Total pendapatan yang ditampilkan program adalah Rp 85000.0.
- Enkapsulasi: field _totalPendapatan bersifat privat karena menggunakan awalan underscore. Nilainya hanya berubah melalui method terimaPaket() dan hanya bisa dibaca melalui getter cekPendapatan, sehingga tidak ada bagian lain yang dapat mengubahnya secara langsung.
- Constructor class anak meneruskan argumen ke constructor class induk dengan super.namaPengirim, super.tujuan, dan super.beratKg, sehingga objek class anak tetap memiliki atribut lengkap seperti class induknya.
- Perulangan while (true) menjaga menu tetap muncul sampai pengguna memilih menu 4. Percabangan if-else if mengarahkan setiap pilihan ke proses yang sesuai, dan break menghentikan perulangan saat pengguna keluar.

6. Kesimpulan

Berdasarkan praktikum ini, konsep inheritance, method overriding, dan enkapsulasi pada bahasa Dart berhasil diterapkan dalam program agen ekspedisi CepatKirim. Class PaketReguler dan PaketKilat mewarisi class Paket serta menimpa method hitungOngkir() sesuai tarif masing-masing. Class AgenEkspedisi menjaga total pendapatan melalui field privat dan getter, sekaligus mengelola daftar paket dengan List<Paket>. Program berjalan sesuai harapan, ditunjukkan oleh hasil perhitungan ongkos kirir dan total pendapatan Rp 85000.0 dari dua paket yang diterima.